Technical Document: Facial Recognition with Emotion and Liveliness

COS30082 Applied Machine Learning, Swinburne University of Technology · October 29, 2026 · Truong Ngoc Gia Hieu

1. Purpose and scope

This document explains how our face system is designed and organised, so every teammate can build and test their own part without asking.

The system looks at a photo or webcam frame, decides whether the face is a real live person, tells us who it is, and reads the emotion. If the person is not known, it offers to add them.

- **In scope:** face verification, emotion recognition, liveness (anti-spoofing), enrolling new people, training and running everything in Google Colab.
- **Out of scope:** mobile apps, tracking several faces at once, production deployment.
- **Status:** the repository has the folder structure and a README per folder; the Python code is not written yet. Numbers marked *proposed* (thresholds, image size, frames per enrollment) are starting points the team must confirm.

2. System overview

Three models share one pipeline, and a liveness check decides whether the other two are allowed to run.

Task	Question	Output	Data folder on Drive
Face verification	Who is this?	ID, name, similarity score	Face Recognition
Emotion recognition	How does the face look?	Emotion ID, then its description	Emotion Detection
Liveness (anti-spoofing)	Is this a real, live face?	live or spoof, score	Anti-Spoofing

Verify, then enroll. Every input is first checked against the identities table. A match above the threshold prints VERIFIED with the person's ID, name and status. No match prints NOT FOUND and offers to add the person as a new member.

Runtime. All Python runs in Google Colab. Data, trained weights and the lookup tables live on Google Drive; code and settings live on GitHub. Colab cannot open a laptop camera directly, so a short JavaScript snippet captures a photo in the notebook, or the user uploads a photo or video.

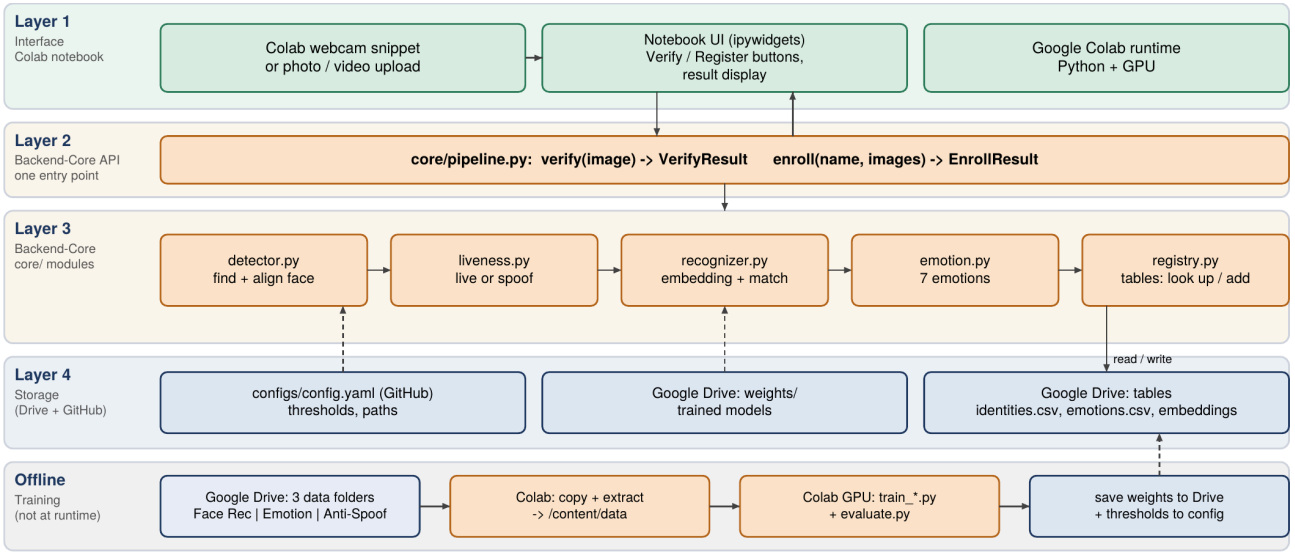
3. Architecture

The system has four runtime layers plus an offline training step. The interface never touches models; it only calls `core/pipeline.py`.

Layer	Where	Job
1. Interface	Colab notebook	Capture a photo, show Verify and Register buttons, display the result
2. API	`core/pipeline.py`	The only entry point: `verify(image)` and `enroll(name, images)`
3. Core modules	`core/`	detector, liveness, recognizer, emotion, registry; one job each
4. Storage	Google Drive + GitHub	Weights, identities and emotions tables, embeddings (Drive); code and `config.yaml` (GitHub)
Offline	Colab GPU	Read the data folders, train, evaluate, save weights and thresholds

Design rules

- One direction of dependency: interface, then pipeline, then modules, then storage.
- Each module takes and returns plain data objects (`schemas.py`), so it can be tested alone.
- Anything that can change without editing code (thresholds, paths, number of frames) lives in `config.yaml`.



4. Repository and branch structure

The repo has two branches: `main` for the final submission and `develop` where the team works. Folders are split so GUI code and model code never mix.

Folder	Purpose
`Documents/`	Assignment brief, reports
`Frontend-GUI/ui/`	Notebook interface: buttons, result display, drawing boxes and labels
`Frontend-GUI/camera/`	Photo capture (webcam snippet) or file upload
`Frontend-GUI/assets/`	Small icons, fonts, styles
`Backend-Core/configs/`	`config.yaml` : thresholds, paths, number of enrollment frames
`Backend-Core/core/`	The pipeline and its modules (section 7)
`Backend-Core/training/`	Data loaders, one train script per task, `evaluate.py`
`Backend-Core/weights/`	Small trained weights (each file under 100 MB)
`Backend-Core/registry_data/`	Local copy or index of enrolled users; real data stays on Drive
`Backend-Core/scripts/`	Helper tools such as `extract_dataset.py` and `demo_cli.py`
`Backend-Core/tests/`	pytest unit tests for `core/`

Rules: never push the dataset or any zip to GitHub; never put GUI code in `Backend-Core` or model code in `Frontend-GUI`. Right now each folder holds only a README; the `.py` files named above are planned.

5. Data on Google Drive

All data lives in one Drive project folder with three data sub-folders, one per task. Nothing from these folders goes to GitHub.

Drive folder	Used for	Notes
`Face Recognition/`	Identity classification and verification training	Contents to be confirmed (identity images, verification pairs)
`Emotion Detection/`	Emotion model	Has `train/` and `test/`
`Anti-Spoofing/`	Liveness model	Contents to be confirmed (live and spoof images)
`outputs/` (proposed)	Weights, `identities.csv` , `emotions.csv` , embeddings, reference photos	Created by the team; not yet on Drive

Working in Colab

1. Mount Drive with `drive.mount('/content/drive')` .
2. Copy the folder you need to `/content/data` and unzip there. Reading thousands of small files directly from Drive is slow; copying one zip and extracting locally is much faster.
3. Train from `/content/data` .
4. Save weights and tables back to Drive `outputs/` before the session ends, because `/content` is wiped when Colab resets.

The helper `Backend-Core/scripts/extract_dataset.py` unzips archives in parallel and can resume after an interruption.

6. Models

Three models, each trained from its own Drive folder. Choices marked *proposed* are starting points to confirm.

Model	Input	Output	Approach (proposed)	Loss / metric
Face recognition	Aligned face, 112x112	512-number embedding	CNN backbone (e.g. ResNet), trained two ways and compared	Softmax cross-entropy vs triplet loss; ROC, AUC, EER
Emotion	Aligned face	Emotion ID (7 classes)	CNN with transfer learning	Cross-entropy; accuracy, macro F1, confusion matrix
Liveness	Face crop	live or spoof + score	Binary CNN classifier	Cross-entropy; APCER, BPCER, EER

Face recognition: two training methods.

- *Classification (softmax):* the network learns to name each training identity. Simple and quick, but it only knows people it was trained on, so the embedding is a by-product.
- *Metric learning (triplet loss):* the network learns that two photos of the same person must be closer than photos of different people. This directly fits matching unseen faces, which is what enrolling new people needs.

Both feed the same backbone. We compare them on the validation pairs and keep the better embedding. At runtime a face becomes an embedding and is compared with each enrolled person using cosine similarity:

$$score(a, b) = (a \cdot b) / (\|a\| \cdot \|b\|)$$

The best score is compared with a threshold chosen from the ROC curve (section 9).

7. Core pipeline

For every image, `pipeline.verify()` runs the modules in a fixed order, and liveness acts as a gate: a spoofed face is never recognised.

1. **detector** finds one face, aligns it and crops it to 112x112. No face, or more than one, stops the run.
2. **liveness** returns live or spoof. Spoof stops the run with the result SPOOF.
3. **recognizer** turns the face into an embedding and searches the identities table.
4. **emotion** predicts an emotion ID for the same live face.
5. **pipeline** merges everything into one result object for the interface.

Module	Responsibility	Reads	Returns
`detector.py`	Detect and align the face	Image	Face crop or an error
`liveness.py`	Real or fake	Face crop, liveness weights	live or spoof, score
`recognizer.py`	Embedding and matching	Face crop, recognition weights, identities table	Best ID, score
`emotion.py`	Emotion ID	Face crop, emotion weights	Emotion ID, confidence
`registry.py`	Look up, add, duplicate check	identities table, embeddings	Row or new ID
`schemas.py`	Shared data classes	n/a	`VerifyResult`, `EnrollResult`
`config.py`, `logger.py`	Load settings, logging	`config.yaml`	Settings, log lines

Because each module only exchanges plain data, one teammate can finish `emotion.py` while another is still on `recognizer.py`, and each can be tested without the rest.

8. Verify, enroll and lookup tables

After training, the system keeps two small tables on Google Drive. Every input is checked against them; only when nothing matches is a new person added.

identities.csv (grows)

Column	Meaning
ID	Unique number, e.g. 0001
Name	Person's name
Image	Path to one reference photo on Drive
Status	`old member` (from the training data) or `new member` (enrolled while running)
embedding_file	Path to the saved embedding; this is what matching really uses

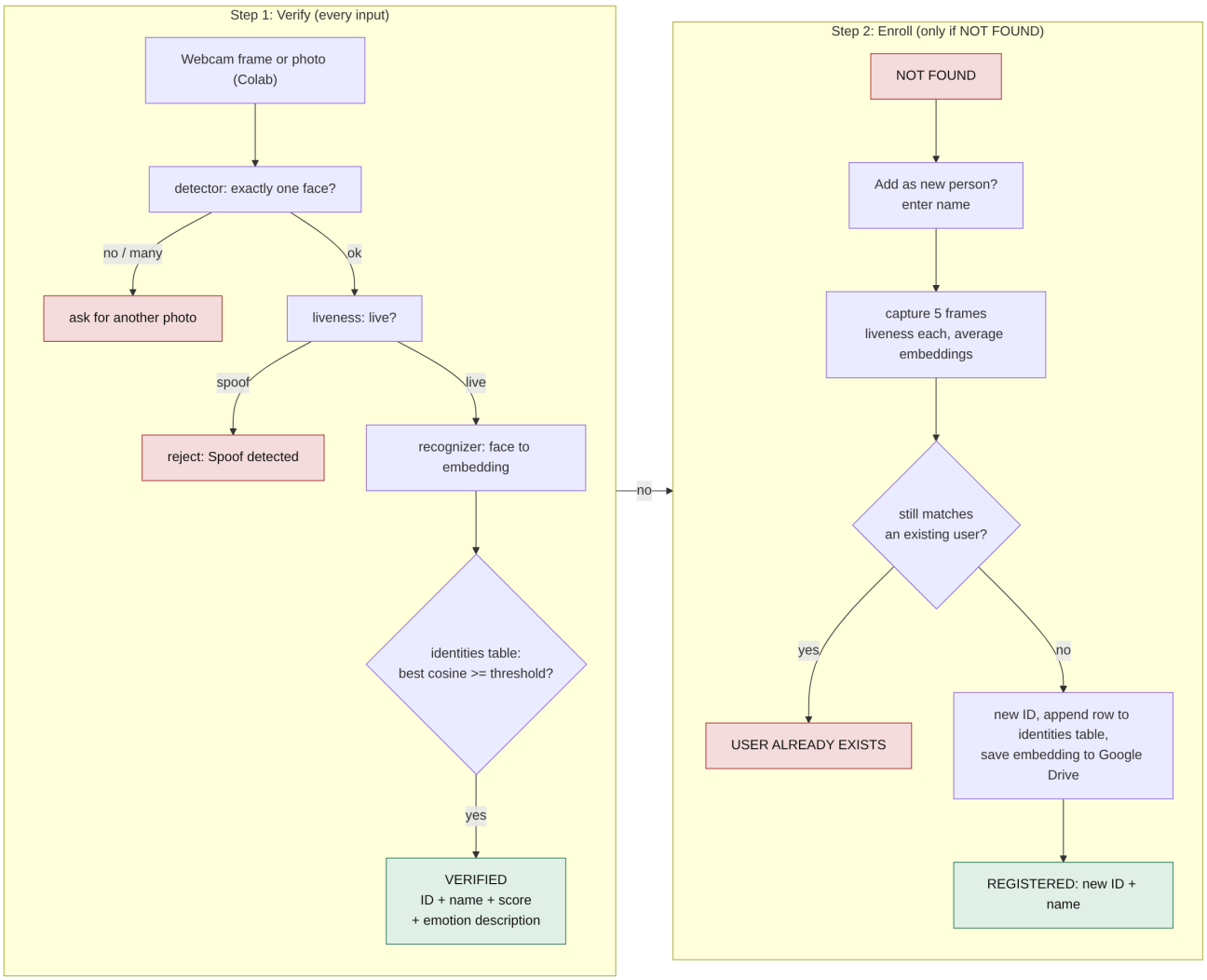
emotions.csv (fixed). Columns ID, label, description, for example 3, happy, smile with raised cheeks. Use the real emotion labels of the dataset.

Verify (every input). Detect, check liveness, make an embedding, find the best cosine score in `identities.csv`. At or above the threshold the result is VERIFIED: ID, Name, Status, score, and the emotion description from `emotions.csv`. Below it the result is NOT FOUND.

Enroll (only after NOT FOUND, and only if the user agrees).

1. Ask for the person's name.
2. Capture 5 frames (proposed). Each must pass the face and liveness checks.
3. Average the embeddings and normalise the result.
4. Search the table again with the stricter `duplicate_threshold`. A match prints USER ALREADY EXISTS and stops.
5. Otherwise create a new ID, append a row with status `new member`, save the embedding and one reference photo to Drive.

Liveness runs before enrollment so a printed photo cannot be registered. Photos and embeddings are personal data: keep them on Drive only, and ask each person for consent before saving their photo.



9. Configuration and thresholds

All tunable values live in `Backend-Core/configs/config.yaml`, so changing a threshold never means editing code. The values below are the placeholders currently in the file; set them from validation results after training.

Setting	Placeholder	Meaning
`verification.metric`	cosine	How two embeddings are compared
`verification.threshold`	0.5	Minimum score to accept a match (VERIFIED)
`verification.duplicate_threshold`	0.6	Stricter score used at enrollment to detect USER ALREADY EXISTS
`liveness.threshold`	0.5	Minimum score to call a face live
`registration.num_frames`	5	Frames averaged when enrolling
`emotion.classes`	7 labels	Must match `emotions.csv`
`paths.*`	data, weights, registry	Where data, weights and tables are read from

How to choose the verification threshold. Score every pair in the validation pairs file, draw the ROC curve, and pick the point at the equal error rate (where false accepts equal false rejects), or a stricter point if wrongly accepting a stranger is worse than rejecting a real user. Write the chosen value back into `config.yaml`.

10. Training and evaluation workflow

Training is offline. Each teammate trains one model in Colab from its own Drive folder, so work does not collide.

Script	Task	Data folder	Saves
`train_classification.py`	Softmax identity classifier	Face Recognition	Recognition weights
`train_metric.py`	Triplet-loss embedding	Face Recognition	Recognition weights
`train_liveness.py`	live vs spoof	Anti-Spoofing	Liveness weights
`train_emotion.py`	7-class emotion	Emotion Detection (`train/` , `test/`)	Emotion weights
`evaluate.py`	Metrics for all models	Validation / test data	Metrics, ROC, thresholds

Steps

1. Clone the GitHub repo in Colab, mount Drive, copy and extract the data you need.
2. Train with a fixed random seed and write each run's settings next to its weights.
3. Evaluate on data the model never saw. Split by person, not by photo, so the same face is not in both train and validation.
4. Save the best weights to Drive `outputs/` , and the chosen thresholds to `config.yaml` .
5. Build the tables: one `identities.csv` row per training identity with status `old member` , plus its embedding and a reference photo; and `emotions.csv` .

Metrics to report: verification ROC, AUC and EER; emotion accuracy, macro F1 and confusion matrix; liveness APCER, BPCER and EER. Record them in the report with the exact threshold used.

11. Team workflow

We use two branches only, to keep things simple.

Branch	Use	Who pushes
`main`	Final submission only	Merge from `develop` by pull request when the project is ready
`develop`	Integration; the team works here	Everyone

Working rules

- Before you start: `git pull origin develop` . Commit small and often with a clear message, such as `Add liveness training script` .
- Work in your own folder or file (for example `training/train_emotion.py`) to avoid merge conflicts. Tell the team before editing shared files such as `config.yaml` or `schemas.py` .
- If two people must touch the same file, pull first, then push soon after.
- Never commit data, zip files, personal photos, embeddings or tables. The `.gitignore` should block `data/` , `*.zip` , `__pycache__/` and `.venv/` .
- Weight files go to GitHub only if under 100 MB; otherwise use Drive.
- Merge `develop` into `main` through a pull request, and only for the submission. Ideally turn on branch protection for `main` in the repo settings.

12. Setup, testing, open questions and glossary

Quick start in Colab

1. Open a new Colab notebook and choose a GPU runtime.
2. Clone the repo: `git clone https://github.com/hieugia090806/Facial_Recognition_with_Emotion_and_Liveliness.git`, then `git checkout develop`.
3. Mount Drive: `from google.colab import drive; drive.mount('/content/drive')`.
4. Copy and extract the data folder you need into `/content/data`.
5. Run your training script, or open the interface notebook to try Verify and Register.

Testing

- Unit tests with pytest in `Backend-Core/tests` for `core/`, for example: enrolling the same face twice must return USER ALREADY EXISTS, and a spoof must never reach the recognizer.
- A command-line demo, `scripts/demo_cli.py`, to run the pipeline on a photo without the notebook interface.
- End-to-end check with a known old member (expect VERIFIED), a stranger (expect NOT FOUND) and a printed photo (expect SPOOF).

Open questions for the team

Question	Why it matters
Is real-time webcam video required, or are single photos enough?	Colab captures one photo at a time; real-time needs a different interface
What exactly is inside `Face Recognition/` and `Anti-Spoofing/` on Drive?	Decides the loaders and the validation pairs
Which emotion labels does the dataset use?	`emotions.csv` and the model's output size
Which backbone, image size and embedding size do we use?	Affects accuracy and training time
Where do `outputs/` (weights, tables) live, and who owns it?	Everyone needs read access; only a few should write
`extract_dataset.py` exists on one laptop only; who adds it to `Backend-Core/scripts/`?	The team needs the same extraction tool

Glossary

Term	Meaning
Embedding	A list of numbers (e.g. 512) that describes a face; similar faces give similar lists
Cosine similarity	A score from -1 to 1 for how alike two embeddings are
Threshold	The score above which we accept a match
Liveness / anti-spoofing	Telling a real face from a photo, screen or mask
ROC, AUC	Curve and area showing the trade-off between false accepts and false rejects
EER	Equal error rate: the point where false accepts equal false rejects
APCER, BPCER	Spoof wrongly accepted, real face wrongly rejected
Enrollment	Adding a new person to the identities table